NumCompute-Stream

Student ID	a3185100
Student Name	Hemanth Kumar Reddyvari
Course	Programming for AI
Assignment	A Modular Ensemble Tree-based Streaming Machine Learning Framework
Git Link	https://github.com/hemanthglobal/numcompute_stream_v2.git

1. Design Decisions

1.1 Framework Overview

NumCompute-Stream extends the original NumCompute toolkit into a streaming, ensemble-capable machine learning framework. It is built only with Python and NumPy. Every component has a partial_fit() method, so the whole pipeline including preprocessing, statistics, metrics, tree models, and ensembles can be updated in chunks as new data arrives. There is no need to re-read earlier data.

1.2 Streaming Architecture

The core streaming rule is simple. Each module takes a chunk, updates its internal state, and returns self. This keeps every component compatible with the Pipeline class. The StreamTrainer wrapper runs the chunk loop and logs per-chunk accuracy, cumulative accuracy, fit time, and retained buffer size. The full history is kept available for visualisation.

1.3 Numerically Stable Running Statistics (Welford + Per-feature NaN Weighting)

StandardScaler and StreamingStats both use a parallel Welford update to maintain a running mean and variance without storing all past data. One important fix was made here. The merge weights now use per-feature non-NaN counts instead of total row counts. Without this, a chunk where column k is fully NaN would still pull the running mean towards zero and corrupt all later transforms. The corrected formula is:

```
n_total = n_prev + n_valid_k
mean_k = (n_prev * mean_k + n_valid_k * chunk_mean_k) / n_total
```

This makes the running mean match np.nanmean exactly over the combined data. It was verified by regression tests that failed on the original code and passed after the fix.

1.4 Bounded Buffer for Sliding-Window Streaming

DecisionTreeClassifier, RandomForestClassifier, BaggingClassifier, and AdaBoostClassifier all accept a max_buffer parameter. When it is set, the retained training buffer is capped to the most recent max_buffer rows after each partial_fit. This changes the design from unbounded batch refit to a real sliding-window stream. Memory then settles at a fixed level instead of growing with the stream. The StreamTrainer memory log shows the actual buffer size, not just the current chunk, so the growth versus plateau behaviour is clearly visible in the dashboard.

1.5 Ensemble Methods: Three Strategies

Method	Strategy	Streaming Behaviour
RandomForestClassifier	Bagging + sqrt feature sampling	Bootstrap refit on retained buffer
BaggingClassifier	Bagging, all features	Bootstrap refit on retained buffer

Method	Strategy	Streaming Behaviour
AdaBoostClassifier	SAMME (multi-class)	Full boosting loop on retained buffer

AdaBoost uses the SAMME algorithm. It adds a log(K-1) correction term to the alpha weight so that the baseline accounts for random guessing across K classes. All three methods are available through the single EnsembleClassifier wrapper using the method parameter, and max_buffer is passed through to them directly.

1.6 Consistent API Design

Every class exposes fit(), partial_fit(), transform() or predict(), and fit_transform(). The Pipeline class chains transformers and a final estimator, and calls partial_fit on each step in order. This follows scikit-learn's design, so any component can be swapped without changing the surrounding code. The visualise.py module is stateless. All its functions take arrays and return matplotlib figures, so it can be used from scripts, notebooks, or pipeline logs in the same way.

2. Testing and Edge Cases

2.1 Test Suite Overview

NumCompute-Stream has 115 unit tests across 8 test files, one per major module group. The tests use pytest and are organised into classes that cover normal operation, boundary conditions, streaming edge cases, and error handling. All 115 tests pass.

Test File	Module(s)	Tests	Key Focus
test_preprocessing.py	preprocessing.py	18	Streaming scalers, NaN handling, encoder growth
test_stats.py	stats.py	17	Running mean/var/min/max, windowed ops, reset
test_metrics.py	metrics.py	17	Streaming confusion matrix, rolling window, AUC
test_tree.py	tree.py	11	Gini/entropy, partial_fit, predict_proba, max_features
test_ensemble.py	ensemble.py	15	RF/Bagging/AdaBoost, partial_fit, proba sums
test_pipeline_stream.py	pipeline.py, stream.py	16	Streaming pipeline, StreamTrainer log, score_chunk
test_nan_regression.py	preprocessing, stats	11	NaN weighting regression (fail to fix to pass)
test_buffer.py	tree, ensemble	9	max_buffer cap, memory plateau, predict correctness
TOTAL		115	

2.2 Systematic Debugging: NaN Regression Tests

The most important tests are in test_nan_regression.py. They were written before the fix was applied, confirmed to fail on the original code, and then seen to pass after the per-feature NaN weighting was added. This order is direct evidence of test-driven debugging:

- `test_partial_fit_nan_mean_matches_nanmean`: running mean must equal np.nanmean exactly.
- `test_partial_fit_nan_var_matches_nanvar`: running variance must equal np.nanvar exactly.
- `test_all_nan_column_stays_zero`: an all-NaN column must not produce inf or nan.
- `test_transform_no_nan_output`: transform on NaN-heavy fit data must stay finite.

- `test_n_seen_is_per_feature_array`: n_seen_ must be a per-feature array after the fix.

2.3 Edge Cases Covered

- **NaN columns:** per-feature non-NaN weighting in all Welford updates prevents corruption.
- **Zero-variance columns:** StandardScaler sets std to 1.0 to avoid division by zero.
- **All-NaN column:** mean and variance stay at 0.0 and remain finite, with no inf or nan passed on.
- **Single-row chunks:** all streaming updates handle n=1 without any special case.
- **Empty rolling window:** windowed_quantile and rolling_accuracy raise RuntimeError if not configured.
- **New class in a later chunk:** the StreamingMetrics confusion matrix expands from 2x2 to 3x3 cleanly.
- **max_buffer smaller than chunk:** the last chunk is retained and predict still returns the correct shape.
- **Transform before fit:** RuntimeError is raised with a clear message in all scalers.
- **Invalid ensemble method:** EnsembleClassifier raises ValueError at once.
- **predict_proba row sums:** verified to equal 1.0 (atol 1e-6) for all three ensemble methods.

3. Benchmark Results

3.1 Streaming Model Performance

All benchmarks were run on a stream of 2,000 samples split into 10 chunks of 200 each. Each timing is the mean of 5 repetitions. The pipeline benchmark uses a StandardScaler and RandomForest(n=10) chain.

Operation	Time (s)	Notes
Single DecisionTree, 10 chunks, n=2,000	9.01	Streaming, max_depth=5
Random Forest n=10, 10 chunks, n=2,000	13.64	Streaming, bootstrap refit
AdaBoost n=10, 10 chunks, n=2,000	24.88	SAMME, max_depth=1 stumps
Pipeline (Scaler + RF), 10 chunks	5.32	Welford scaler adds minimal overhead
NumPy mean (n=100,000)	0.0000	vs Python loop 0.0049s, 273x faster

3.2 Streaming Accuracy Comparison

Model	Streaming Accuracy	Notes
Single DecisionTree	98.30%	Full accumulation, max_depth=5
Random Forest (n=10)	91.50%	Bootstrap sampling introduces variance

3.3 Analysis

The single tree gives higher streaming accuracy (98.3%) than the forest (91.5%) on this synthetic dataset. This is expected. With unlimited buffer accumulation the single tree fits the full 2,000-sample history, while each forest tree trains on a bootstrap sample of the same data. This adds variance on purpose, which helps on real-world noisy datasets but can slightly underfit a clean synthetic one. AdaBoost is the slowest because it re-runs the full boosting loop of 10 weighted refit passes on the accumulated data at each chunk. The Pipeline benchmark confirms that the Welford scaler adds very little overhead. The 5.32s pipeline time is faster than the raw RF mainly because the

pipeline benchmark uses n_estimators=5 instead of 10. The 273x NumPy speedup over Python loops repeats Assignment 2.1's finding and confirms that all core computations stay vectorised.

4. Reflections

4.1 What Was Technically Challenging

- **NaN-weighted Welford update.** The original code used the total row count as the merge weight, which quietly corrupted running estimates whenever a chunk had column-wise NaNs. Writing the regression test first, watching it fail, and then applying the per-feature count fix made both the bug and the solution concrete and verifiable.
- **Partial_fit strategy for trees.** True online tree learning, such as Hoeffding trees, is hard to implement correctly. The chosen design accumulates data and refits, with an optional cap using max_buffer. It is honest about what it does, numerically correct, and far easier to test than incremental node splitting. The max_buffer option turns it into a real sliding-window stream.
- **SAMME AdaBoost.** The multi-class extension needs the log(K-1) term in the alpha weight to account for the K-class random guessing baseline. Getting this right meant reading the original Freund and Schapire paper rather than relying on intuition from binary AdaBoost.
- **Pipeline partial_fit chain.** Each intermediate step's partial_fit had to update its state before transforming the data, so the scaler fitted on chunk 1 transforms chunk 2 with the correct running statistics. This needed careful ordering inside the pipeline loop.

4.2 What This Assignment Taught

- Streaming ML is not just batch ML called again and again. NaN handling, memory management, and state consistency need deliberate design at every layer.
- Test-driven debugging, where you write a failing test, apply the fix, and watch it pass, is more reliable than writing tests after the fact.
- The API consistency lesson from Assignment 2.1 paid off directly. Because every component already used fit and transform, adding partial_fit was an addition rather than a rewrite.
- Accurate docstrings matter. Phrases like online growth implied Hoeffding-tree behaviour that the code did not have. Rewriting it to chunk-wise refit on retained buffer is both accurate and more informative.

4.3 What Would Be Improved with More Time

- Replace the accumulated-data refit with a true Hoeffding (VFDT) tree for genuine O(1) memory per chunk, which would allow infinite streams.
- Add concept drift detection (ADWIN or Page-Hinkley) to reset the model automatically when the data distribution shifts.
- Extend the visualise.py dashboard to include a live rolling-window accuracy plot next to the cumulative accuracy, so drift becomes visible at once.